

TALLER PRACTICO - BLOQUE 5

De clases aisladas a objetos que colaboran - SmartLibrary

Diseño de Software - IV Semestre | Interfaces - Asociacion - Agregacion - Composicion - Herencia - Componentes UML

Integrantes: Jose Nicolas Mora y Drako David Salazar - Ingenieria de Software (IV semestre) | **Fecha:** Octubre 2026
| **Herramientas:** Draw.io (boceto) + Java + VS Code

Punto de partida. Partimos del diseño del Bloque 4 (estudiantes, libros, prestamos y reservas) y sumamos los requisitos nuevos: **R10** un libro tiene varios ejemplares y cada ejemplar es de un solo libro; **R11** el prestamo guarda renovaciones con fecha anterior, nueva fecha y fecha de renovacion; **R12** hay usuarios con datos comunes (identificacion, nombre, correo) y datos propios de estudiante y bibliotecario; **R13** algunos usuarios reciben notificaciones; **R14** agrupamos todo en 4 gestiones: Usuarios, Catalogo, Prestamos y Reservas. Nuestra regla fue no poner composicion o herencia por pereza, sino solo si el requisito lo pide.

1. Actividad 1 - Diagnostico de relaciones (15 min)

Pareja	Que significa en el problema	Existencia independiente?	Decision inicial
Estudiante - Prestamo	El estudiante pide prestado. Se conocen para registrar quien se lleva que.	Si. El estudiante existe sin prestamos y el prestamo ya creado queda como historial.	Asociacion
Libro - Ejemplar	El libro es la idea general y el ejemplar es el libro fisico de la estanteria.	Si, a medias. Si borro el libro del catalogo, el ejemplar fisico sigue ahi. Por eso no lo vemos como parte que muere.	Agregacion
Prestamo - Renovacion	La renovacion es una anotacion dentro del historial del prestamo.	No. Una renovacion sola no significa nada, solo vale dentro de su prestamo.	Composicion
Usuario - Estudiante	Estudiante es una forma de ser usuario dentro de la biblioteca.	No son independientes, es el mismo objeto visto mas especifico.	Herencia (es-un)
Usuario - Bibliotecario	Bibliotecario tambien es un usuario, pero con rol de empleado.	Igual, es el mismo objeto especializado.	Herencia (es-un)

1. Que dos clases se relacionen significa que una sea parte de la otra? No. Por ejemplo Estudiante-Prestamo solo se conocen, ninguno es parte del otro. Para ser todo-parte debe haber sentido de pertenencia, como Prestamo-Renovacion.

2. Que evidencia usamos? Leimos R10, R11 y R12 palabra por palabra. Si el requisito dice 'cada renovacion conserva...' entendemos que vive dentro del prestamo. Si dice 'un libro puede tener varios ejemplares' entendemos que el ejemplar puede existir aparte.

3. Cual seria incorrecto decidir solo por nombres? Libro-Ejemplar, porque suenan a que uno contiene al otro y uno diria composicion rapido, pero cuando pensamos en el ciclo de vida real vimos que es agregacion. Tambien Usuario-Estudiante, porque compartir atributos no basta para heredar.

2. Actividad 2 - Asociacion, agregacion y composicion (25 min)

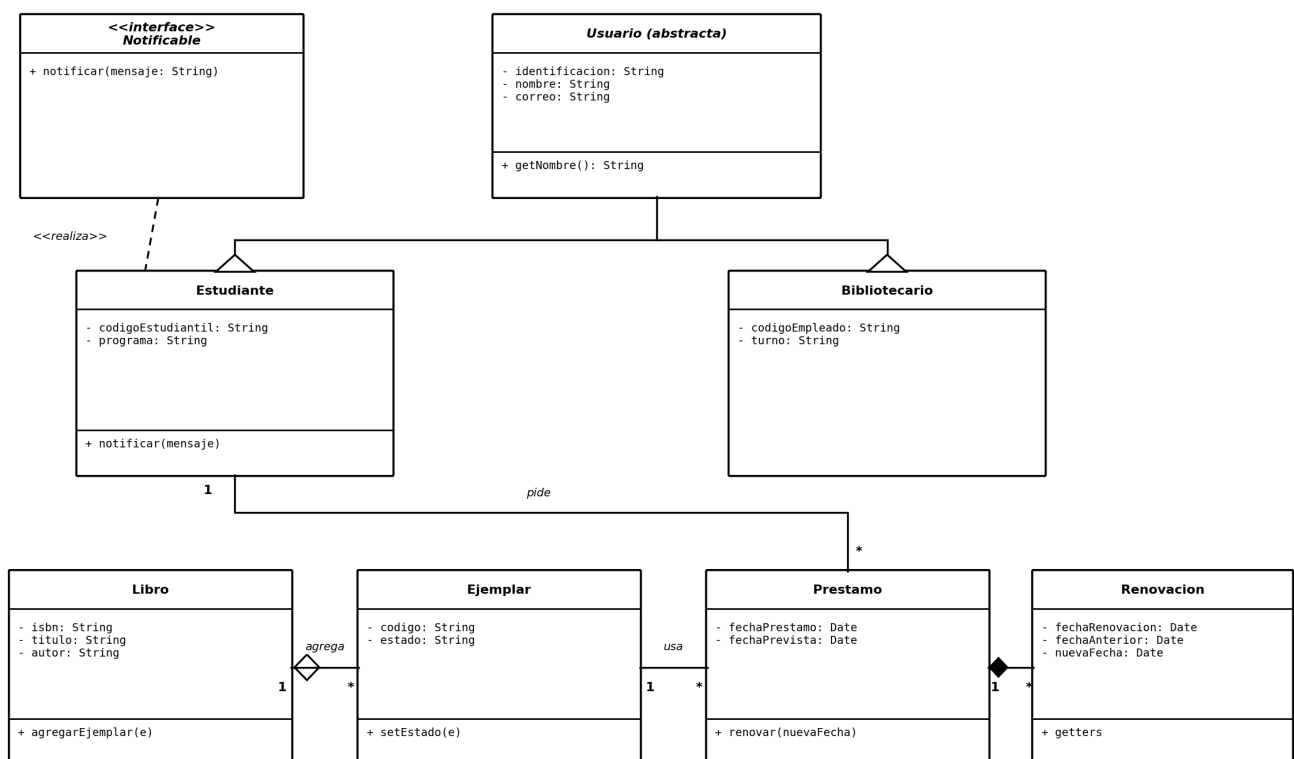
Hicimos el fragmento con **Libro, Ejemplar, Estudiante, Prestamo y Renovacion** con sus multiplicidades. La imagen del diagrama completo va abajo en la Figura 1. Aqui resumimos la decision:

Relacion	Tipo elegido	Multiplicidad	Justificacion en el dominio
Estudiante - Prestamo	Asociacion	1 a *	Solo se conocen. El estudiante sigue existiendo sin prestamos y el prestamo necesita al estudiante pero no lo contiene. Si se va el estudiante, el prestamo queda como registro.
Prestamo - Ejemplar	Asociacion	* a 1	El prestamo usa un ejemplar, pero el ejemplar sigue existiendo antes y despues. Un mismo ejemplar se presta muchas veces en el tiempo.
Libro - Ejemplar	Agregacion	1 a *	Es todo-parte pero debil: el ejemplar necesita un libro para tener sentido en el catalogo, pero si elimino el libro del sistema el objeto fisico no se destruye. Supuesto: en nuestra biblioteca no se desechan los libros si se borran del catalogo.
Prestamo - Renovacion	Composicion (rombo negro)	1 a *	La renovacion solo existe como historial interno. Si borro el prestamo, sus renovaciones pierden todo significado. No se comparten con otros prestamos.

Criterio que usamos: primero preguntamos si solo se conocen (asociacion). Despues si hay todo-parte. Despues si la parte vive sola (agregacion) o muere con el todo (composicion). **Reto:** si otro equipo pone Libro-Ejemplar como composicion tambien vale, siempre que diga el supuesto, por ejemplo 'en nuestra sede si se da de baja el libro se desechan los ejemplares'. Nosotros dejamos el supuesto escrito arriba.

Figura 1 - Diagrama de clases actualizado

SmartLibrary - Diagrama de Clases (Bloque 5)



Asociacion: linea simple | Agregacion: rombo blanco | Composicion: rombo negro | Herencia: triangulo | Realizacion: punteada

Clases: Usuario (abstracta), Estudiante, Bibliotecario, Libro, Ejemplar, Prestamo, Renovacion + Notificable. Rombo blanco = agregacion, negro = composicion.

3. Actividad 3 - Herencia: es-un o solo se parecen? (20 min)

Evaluamos la propuesta de generalizacion: Usuario como superclase de Estudiante y Bibliotecario, donde Usuario tiene identificacion, nombre y correo.

4. Estudiante es realmente un Usuario? Si. En la biblioteca todo estudiante que pide libros actua como usuario: se identifica, tiene correo y hace prestamos y reservas. Puedo decir 'el estudiante es un usuario' sin que suene raro y lo puedo pasar donde pidan un Usuario.

5. Bibliotecario es realmente un Usuario? Si. Tambien se identifica en el sistema, tiene nombre y correo, solo que su rol es atender. Es otro tipo de usuario, no un objeto distinto. Por eso la herencia tiene sentido y no es solo para no repetir codigo.

6. Basta con que se repitan 3 atributos para crear Usuario? No. Repetir atributos es una pista, pero lo que importa es que haya un concepto real. Si solo copio atributos por pereza puedo unir cosas que no son lo mismo. Aqui si hay concepto: 'persona registrada en SmartLibrary' con reglas comunes.

7. Ejemplo de mala herencia solo por ahorrar atributos: si Libro y Estudiante tienen 'codigo' y 'fecha de registro' no voy a crear una superclase CosaConCodigo para ellos, porque un libro no es un estudiante. Quedaria raro y despues no sabria donde poner metodos como prestar() o renovar(). Otro ejemplo: juntar Bibliotecario y Ejemplar porque ambos tienen 'codigo' seria un error total.

Decision: si dejamos la generalizacion. Usuario queda **abstracta** porque nunca creamos un usuario generico, siempre es estudiante o bibliotecario. Asi el diagrama queda mas limpio y el codigo reutiliza el constructor.

4. Actividad 4 - Interfaces: contrato de notificacion (20 min)

Creamos la interfaz: `public interface Notificable { void notificar(String mensaje); }`

8. Quien implementa Notificable? Solo **Estudiante**. Porque R13 dice 'algunos usuarios pueden recibir notificaciones' y en nuestro caso los avisos son 'su prestamo fue renovado' o 'su reserva esta lista', que solo le importan al que pide prestado. El Bibliotecario lo dejamos sin implementar a proposito para mostrar que el contrato es selectivo. Si manana el bibliotecario necesita avisos, solo le agregamos `implements Notificable` sin tocar la herencia.

9. UML: en la Figura 1 se ve con linea punteada de Estudiante a Notificable con <<realiza>>. En el codigo es `class Estudiante extends Usuario implements Notificable`.

10. Que garantiza y que NO: garantiza que el objeto tiene un metodo `notificar(String)` que puedo llamar sin saber que clase es. NO dice como lo envia (consola, correo, WhatsApp), ni cuando se llama, ni que guarda. Eso lo decide cada clase.

11. Por que interfaz no es superclase: la superclase me da atributos y codigo heredado (identificacion, nombre) y es un 'es-un'. La interfaz solo me obliga a cumplir un comportamiento, es un 'puede-hacer'. En Java solo puedo heredar de una clase, pero puedo implementar varias interfaces.

Pregunta	Respuesta del equipo
Que promete una clase que implementa Notificable?	Que se le puede llamar a <code>notificar(mensaje)</code> y va a responder, aunque cada uno lo haga a su manera.
La interfaz define como se envia el mensaje?	No. Solo define el nombre del metodo y el parametro. Nosotros lo hicimos con <code>System.out.println</code> para probar, despues podria ser correo.
Puede heredar de Usuario e implementar Notificable a la vez?	Si. Estudiante lo hace. Una cosa es de quien hereda los datos y otra que contratos cumple. Java lo permite: <code>extends uno + implements varios</code> .

5. Actividad 5 - Implementacion en Java (25 min)

Solo hicimos el pedazo necesario, no toda la app. Clases: **Usuario (abstracta)**, **Estudiante**, **Bibliotecario**, **Renovacion**, **Prestamo** mas **Libro**, **Ejemplar**, **Reserva** y **Main** para probar. Todo esta en la carpeta `src/` del repo. La regla importante esta en `Prestamo.renovar()`: si la nueva fecha no es posterior a la vigente, lanza `IllegalArgumentException` y no cambia

nada. Así protegemos R11. Hecho por Jose Nicolas Mora y Drako David Salazar.

```
Fragmento clave (Prestamo.renovar): if (!nuevaFecha.isAfter(fechaPrevista)) throw new
IllegalArgumentException(...); Renovacion r = new Renovacion(LocalDate.now(), fechaPrevista, nuevaFecha);
renovaciones.add(r); fechaPrevista = nuevaFecha; Y en Estudiante: public void notificar(String m){
System.out.println("[Notificacion para "+getNombre()+"]": "+m); }
```

Evidencia de ejecucion (salida real de la consola)

Compilamos y ejecutamos así: `javac -d out src/*.java` y `java -cp out Main`. Esta es la salida tal cual, con las líneas largas divididas solo para que se lean completas:

=== PRUEBA SMARTLIBRARY BLOQUE 5 ===
Prestamo creado: Prestamo de EJ-001 a Jose Nicolas Mora devuelve: 2026-10-08
Fecha prevista inicial: 2026-10-08

--- Prueba 1: renovacion VALIDA al 2026-10-15 ---
[Notificacion para Jose Nicolas Mora]: Su prestamo fue renovado.
Nueva fecha: 2026-10-15
Cantidad de renovaciones: 1
Historial: Renovacion: 2026-10-08 -> 2026-10-15 (hecha el 2026-09-30)

--- Prueba 2: renovacion INVALIDA al 2026-10-10 ---
OK, el sistema la rechaza: Renovacion invalida: la nueva fecha (2026-10-10) debe ser posterior a
la fecha prevista vigente (2026-10-15).
La fecha sigue siendo: 2026-10-15
Cantidad de renovaciones sigue en: 1

=== FIN DE LA PRUEBA ===

Prueba 1 (valida): renovamos del 08-oct al 15-oct-2026, se creo la Renovacion, se actualizo la fecha y se notifico al estudiante. **Prueba 2 (invalida):** intentamos renovar al 10-oct que es anterior al 15-oct vigente; el sistema la rechazo con excepcion, la fecha no cambio y el conteo siguio en 1. Eso prueba que la regla quedo protegida.

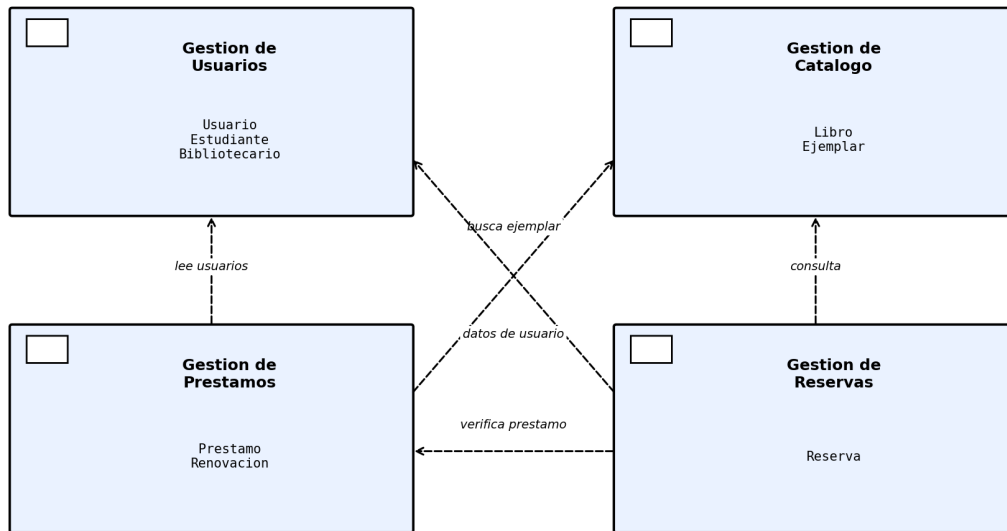
6. Actividad 6 - Vista funcional con componentes UML (10 min)

No es arquitectura de microservicios, solo agrupamos las clases por la tarea que hacen (R14):

Componente	Clases relacionadas	De que otro necesita info? Para que?
Gestion de Usuarios	Usuario, Estudiante, Bibliotecario	De nadie casi. Es la base. A veces pide a Prestamos si el usuario tiene multas.
Gestion de Catalogo	Libro, Ejemplar	De Usuarios para saber quien registra el libro (bibliotecario).
Gestion de Prestamos	Prestamo, Renovacion	De Usuarios (quien pide) y de Catalogo (que ejemplar esta disponible). Sin esos no puede prestar.
Gestion de Reservas	Reserva	De Usuarios, Catalogo y Prestamos: para saber si el libro esta prestado y avisar con Notificable cuando se libere.

Figura 2 - Diagrama de componentes

SmartLibrary - Vista funcional por Componentes (Bloque 5)



Vista funcional de alto nivel: no son microservicios, solo agrupan clases por tarea.

Cuatro componentes funcionales y sus dependencias punteadas. Prestamos es el que mas necesita de otros.

Conclusion

Antes veíamos clases sueltas y ahora entendemos que lo importante es como colaboran. Aprendimos a no usar composicion por todo, a que la herencia solo vale si es un verdadero es-un y a que la interfaz es un contrato que no dice como se hace. El prestamo protege su regla de renovacion y los componentes nos ayudan a ordenar el sistema por funciones. Diseñar relaciones nos obliga a leer bien los requisitos y a justificar cada decision.

(124 palabras. Equipo: Jose Nicolas Mora y Drako David Salazar.)

Lista de verificacion

- [x] No use composicion solo porque una clase contiene otra.
- [x] Justifique cada relacion con el dominio (R10-R13).
- [x] La herencia Usuario-Estudiante-Bibliotecario si es es-un.
- [x] Se explicar heredar (extends) vs implementar (implements).
- [x] Las multiplicidades cuadran con los requisitos.
- [x] El codigo refleja el UML.
- [x] Prestamo bloquea renovaciones invalidas.
- [x] Componentes son grupos funcionales, no microservicios.
- [x] No meti GRASP, SOLID ni patrones (eso es del otro bloque).